Relational vs. Graph Databases

Technical Comparison

Jacopo Masia 1842884

March 4, 2026

Outline

0

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

Table of Contents

- ▶ Data Model, Schema & Storage Layout
- ▶ Query Languages: SQL vs. Cypher
- ▶ Query Complexity Analysis
- ▶ Consistency, ACID & the CAP Theorem
- ▶ Scaling
- ▶ Experimental Analysis: Recipes Dataset
- ▶ Conclusions

Formal Data Models

Relational Model

A database is a finite set of **relations**:

$$R \subseteq D_1 \times D_2 \times \cdots \times D_n$$

where each D_i is a domain (type).

- Tuple $t \in R$, Arity n
- Primary key K: minimal superkey
- Foreign key: referential integrity
- Normal forms: 1NF $\rightarrow$ BCNF

Property Graph Model

A graph $G = (V, E, \lambda, \mu)$ where:

- V — set of **nodes** (vertices)
- $E \subseteq V \times V$ — set of **edges**
- $\lambda : V \cup E \rightarrow \Sigma$ — label function
- $\mu : (V \cup E) \times K \rightarrow \mathcal{V}$ — property map

Schema: Predefined vs. Schema-Optional

Rigid Schema (DDL)

- Schema defined *before* data insertion
- `ALTER TABLE` is expensive at scale
- Strong typing: `INT, VARCHAR, DATE...`
- Schema enforced at write time
- Migrations require careful versioning

Benefit: data contract guarantees, easier query optimisation

Schema-Optional (Graph)

- Nodes / edges can have *arbitrary* properties
- Labels act as soft types
- Neo4j supports *schema constraints* optionally
- Adding properties requires no migration
- Heterogeneous graphs natively supported

Benefit: agile modelling, evolving domains

Storage Layout

Row / Columnar Store

Row-oriented (OLTP) — tuple contiguous in page:

id	name	age	city
1	Alice	30	Rome
2	Bob	25	Milan

Column-oriented (OLAP) — one array per column:

```
age[]:    [30, 25, 41, ...]
city[]:   [Rome, Milan, ...]
```

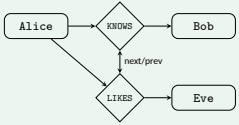
- Compression ratio ↑ on repeated values
- Analytics scan only needed columns
- Examples: Redshift, DuckDB, Parquet

Native Graph Storage (Neo4j)

Fixed-size records on disk:

- Node record: **15 B** Rel record: **34 B**

Index-free adjacency — each node holds a pointer to its *first* relationship; rels form a doubly-linked list:



- Each node holds a **direct pointer** to its first relationship record
- Properties in a **separate property store**

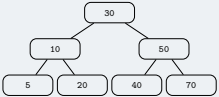
Index Structures

1 Data Model, Schema & Storage Layout

Relational DB — B-Tree Index

B-Tree (default in PostgreSQL, MySQL):

- Balanced tree over a **column's values**
- Lookup, insert, delete: $O(\log n)$
- Supports equality *and* range queries



Graph DB — Index-Free Adjacency

No index needed for traversal:

- Neighbour lookup: $O(1)$ — just follow a pointer
- Cost depends on **neighbourhood size**, not graph size

Indexes still used for entry points:

- **B-Tree / Hash** — find node by property value
- **Full-text (Lucene)** — text search on properties

RDB: index resolves **every JOIN** — $O(\log n)$ per join, globally.
Graph DB: index only for the **entry point** — hops are $O(1)$ pointers.

Table of Contents

2 Query Languages: SQL vs. Cypher

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

SQL — Structured Query Language

Find all recipes that contain “Olive Oil” as an ingredient:

```
SELECT r.name, r.cuisine, r.prep_time
FROM   recipes r
JOIN   recipe_ingredients ri ON ri.recipe_id = r.id
JOIN   ingredients i  ON i.id = ri.ingredient_id
WHERE  i.name = 'Olive Oil'
ORDER  BY r.prep_time ASC;
```

Multi-hop: recipes sharing 2+ ingredients with a given recipe:

```
WITH target_ings AS (
  SELECT ingredient_id FROM recipe_ingredients WHERE recipe_id = 42
)
SELECT r2.name, COUNT(*) AS shared
FROM   recipe_ingredients ri
JOIN   target_ings t  ON t.ingredient_id = ri.ingredient_id
JOIN   recipes r2     ON r2.id           = ri.recipe_id
WHERE  ri.recipe_id <> 42
GROUP  BY r2.id, r2.name
HAVING COUNT(*) >= 2
ORDER  BY shared DESC;
```

Cypher — Graph Query Language (openCypher / GQL)

Same query — recipes containing “Olive Oil”:

```
MATCH  (r:Recipe)-[:CONTAINS]->(i:Ingredient {name: 'Olive Oil'})
RETURN r.name, r.cuisine, r.prep_time
ORDER  BY r.prep_time ASC
```

Multi-hop: recipes sharing 2+ ingredients:

```
MATCH  (target:Recipe {id: 42})-[:CONTAINS]->(i:Ingredient)
       <-[:CONTAINS]-(other:Recipe)
WHERE  other <> target
WITH   other, COUNT(i) AS shared
WHERE  shared >= 2
RETURN other.name, shared
ORDER  BY shared DESC
```

Query Language Comparison

Key differences

- Cypher uses **ASCII-art patterns**: `(node)-[:REL]->(node)`
- No explicit JOIN — traversal is *implicit* in the pattern
- Variable-length paths: `-[:KNOWS*1..5]->` (impossible in plain SQL)

Table of Contents

3 Query Complexity Analysis

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

Query Complexity: Multi-Hop Joins

Relational

k-hop query requires k self-JOINs on the edge table.

Assuming hash-join, each JOIN costs $O(n)$ where $n = |E|$.

Total: $O(k \cdot n)$ — grows with graph size, not neighbourhood size.

With index on foreign keys: $O(k \cdot d)$ per lookup (d = avg degree), but planning overhead for large k remains.

Graph

Traversal cost proportional to **neighbourhood size**, not $|V|$ or $|E|$.

k-hop traversal: $O(d^k)$ where d is average degree.

For sparse graphs ($d \ll n$): dramatically faster.

For dense super-nodes (e.g. hubs): same bottleneck. Strategies: relationship type filtering, degree caps.

Query Complexity: Aggregations & Analytics

Relational — Aggregation Strength

- `GROUP BY` highly optimised (sort-merge, hash-agg)
- Window functions (`OVER PARTITION BY`) efficient
- Columnar layout ⇒ vectorised execution
- Cost-based optimiser rewrites complex queries
- Materialised views for pre-aggregation

Best for: OLAP, reporting, large flat aggregations

Graph — Aggregation Limitations

- `COUNT`, `SUM`, `COLLECT` available in Cypher
- Less mature query optimiser than mature RDBMS
- No columnar layout ⇒ row-at-a-time aggregation
- Graph algorithms (PageRank, BFS, community detection) via **GDS library**
- APOC procedures extend analytical power

Best for: topology-aware aggregations (centrality, clustering)

Complexity Summary Table

Query Type	Relational	Graph DB
Point lookup (PK)	$O(\log n)$	$O(1)$
1-hop neighbour	$O(\log n)$ + JOIN	$O(d)$
k**-hop traversal**	$O(k \cdot n)$	$O(d^k)$
Shortest path	$O(n^2)$	$O((\lvert V\rvert + \lvert E\rvert) \log \lvert V\rvert)$
Global aggregation	$O(n)$	$O(n)$

Table of Contents

4 Consistency, ACID & the CAP Theorem

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

ACID vs. BASE

Relational DBs — Strict ACID

- **A**tomicity: commit entirely or roll back
- **C**onsistency: integrity constraints always hold
- **I**solation: concurrent txns behave serially
- **D**urability: committed data survives crash

PostgreSQL implementation:

- WAL (Write-Ahead Log) for durability
- SSI — Serializable Snapshot Isolation
- Predicate locks to detect conflicts

Graph DBs — ACID or BASE?

Single-instance (e.g. Neo4j):

- Fully **ACID** — WAL-based durability
- Snapshot isolation by default
- Serializable transactions available

Distributed (e.g. Amazon Neptune, JanusGraph):

- Relax to **BASE** for scalability
- **BA**sically Available — always responds
- **S**oft state — may differ across nodes
- **E**ventually consistent — converges over time

Relational DBs **always** target ACID. Graph DBs choose: **ACID** (single-node) or **BASE** (distributed).

The CAP Theorem

CAP

A distributed system can guarantee at most **2 of 3**:

- **C**onsistency — every read returns the latest write
- **A**vailability — every request gets a response
- **P**artition tolerance — system works despite network splits

In practice, *P* is unavoidable ⇒ choose **CP** or **AP**.

PACELC: even without partitions, there is a latency–consistency tradeoff.

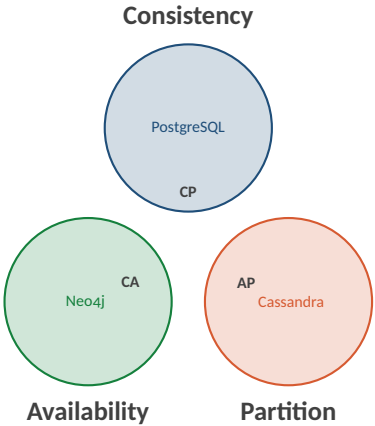


Table of Contents

5 Scaling

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

Vertical vs. Horizontal Scaling

5 Scaling

Relational DB Scaling

Vertical (Scale-Up):

- + More RAM ⇒ larger buffer pool, fewer disk reads
- + Faster CPU / NVMe ⇒ lower query latency
- – Expensive, single point of failure

Horizontal (Scale-Out):

- + **Read replicas** — offload reads, async replication
- + **Sharding** — range/hash partition on shard key
- – Cross-shard JOINs ⇒ expensive distributed queries
- – Distributed txns require **2PC**
- – Stale reads on replicas (eventual consistency)

Graph DB Scaling

Vertical (Scale-Up):

- + Entire graph on one instance — no edge-cuts
- + OS page cache keeps hot subgraphs in RAM
- – Very large graphs (10^{10}+ edges) won't fit

Horizontal (Scale-Out):

- + **Read replicas** — Neo4j Causal Cluster
- – **Graph partitioning is NP-hard** in general, sharding is difficult
- – Edge-cuts force cross-shard pointer follows
- – Loses the $O(1)$ adjacency guarantee

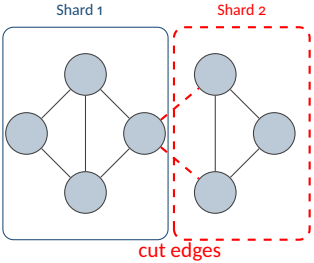
Graph Partitioning & Network Latency

5 Scaling

Why graph partitioning is hard

Balanced graph partitioning (minimise cut edges) is NP-complete. **Impact of edge cuts:**

- Each cross-shard hop $\approx$ 0.5–5 ms network RTT
- k-hop query across s shards: $O(k \cdot s \cdot \text{RTT})$
- Hotspot nodes (hubs) create network bottlenecks



Mitigation strategies

Locality-aware partitioning, edge replication, workload-specific clustering (community detection before partition).

Table of Contents

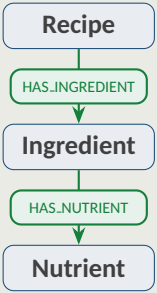
6 Experimental Analysis: Recipes Dataset

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

Dataset Description

6 Experimental Analysis: Recipes Dataset

Graph Schema



Dataset Sizes Tested

Scale	Recipes	Ingredients	Nutrients	R-I Edges	I-N Edges
25%	∼42.5k	4805	142	∼373k	∼304k
50%	∼85k	4869	142	∼746k	∼308k
75%	∼127.5k	4870	142	∼1.12M	∼308k
100%	∼170k	4870	142	∼1.5M	∼308k

Queries Tested

10 queries across 4 categories

Multi-hop (2-hop traversals):

- Nutrients for a recipe
- Recipes sharing an ingredient
- Filter mid-traversal by nutrient amount

Aggregation:

- Total nutrient in recipe
- Top ingredients by nutrient
- Recipes by prep time

Mixed:

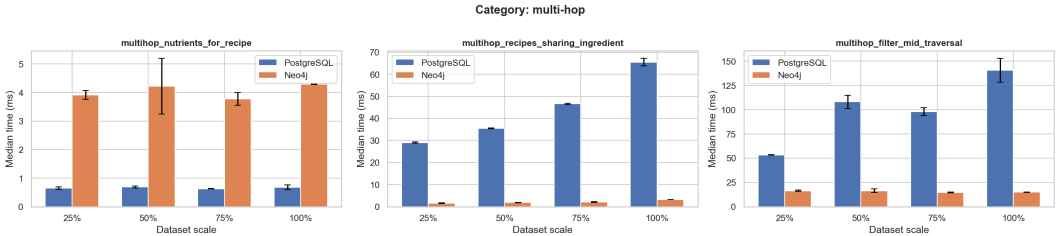
- Most used ingredients
- Subset matching given recipe
- Nutritional density

Variable-length:

- Reachable recipes in N-hops

Results: multihop queries results

6 Experimental Analysis: Recipes Dataset

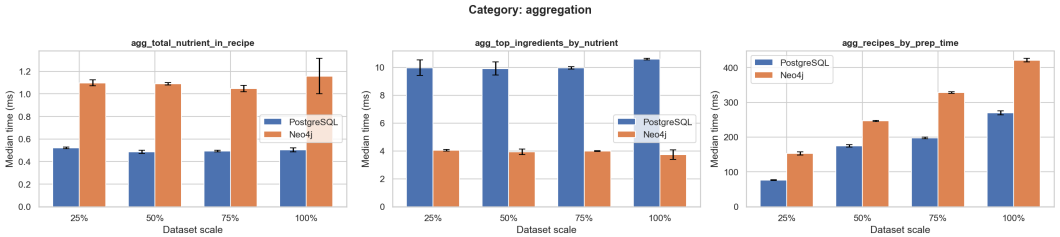


Results: multihop queries analysis

- **Nutrients for a recipe**: shallow traversal, pattern `Recipe → Ingredient → Nutrient` with specific starting recipe. Very fast queries in both cases, we are measuring the fixed per-query overhead of the two DBs, Postgres is faster.
- **Recipes sharing an ingredient**: we notice the dramatic win of Neo4j as expected, the pattern `Recipe → Ingredient ← Recipe` generates very large intermediate results (fan-out) that grows with the dataset size, meanwhile in Neo4j simply follows the pointers.
- **Filter mid-traversal by nutrient amount**: the query is to find recipes containing an ingredient with more than X amount of a given nutrient. Pattern `Recipe → Ingredient → Nutrient` with specific nutrient and amount requires SQL to make a join of the two relations across the full dataset, in Neo4j it's possible to apply the filter on the amount during the traversal.

Results: aggregation queries results

6 Experimental Analysis: Recipes Dataset

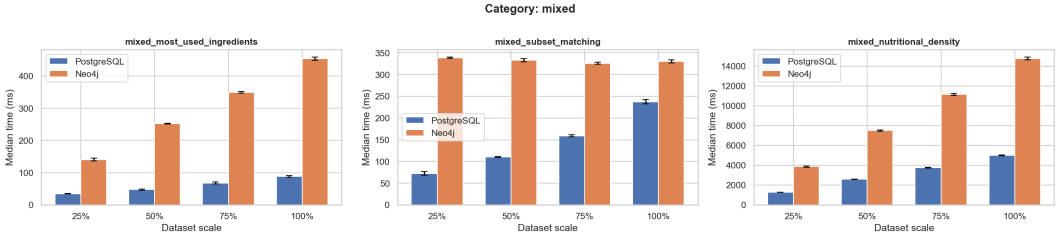


Results: aggregation queries analysis

- **Total nutrients in recipe**: simple sum with selective filter on one recipe, again we are measuring the overhead.
- **Top ingredients by nutrient**: global aggregation that requires SQL to make a full scan of the `has_nutrient` relation filtered by `nutrient_id`, even with an index on `ingredient_id` produces a very large intermediate result, meanwhile in Neo4j a traversal starts from the `Nutrient` node and reaches all the connected Ingredient nodes, a sequential scan of the local neighborhood which is cache-friendly.
- **Recipes by prep time**: range filter on a property followed by a count. Postgres uses a B-tree index on `recipe.minutes` which is ideal for range queries. Neo4j instead is not optimized for property-based range filtering which require a full node scan of all the `Recipe` nodes.

Results: mixed queries results

6 Experimental Analysis: Recipes Dataset

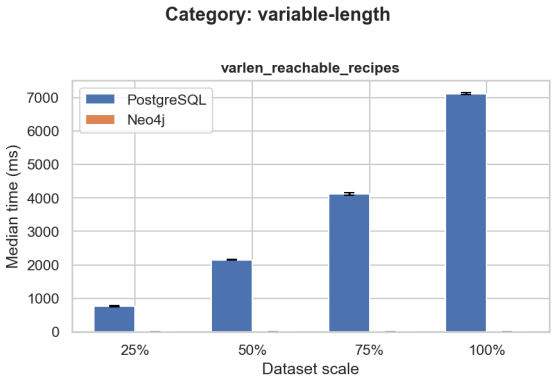


We can see that

Results: mixed queries analysis

- **Most used ingredients**: classic hash aggregation on the `has_ingredient` junction table, Postgres is extremely optimized for this, Neo4j instead must traverse every `Recipe → Ingredient` relationship in the graph without pruning.
- **Subset matching given recipe**: find recipes that contain all ingredients of a given recipe. Double `NOT EXISTS` query for postgres which is optimized for that using anti-joins. Neo4j must check every required ingredient (N relationship existence) for each candidate recipe. We notice though that the computation time is constant for all dataset scales, which suggests that the cost may be dominated by a fixed overhead function (such as `collect()` or `ALL()`).
- **Nutritional density**: a 3-hop aggregation `Recipe→Ingredient→Nutrient` over the entire dataset with `AVG` and `COUNT DISTINCT`. Both systems are slow because the aggregation is global and the dataset is big, but Postgres wins because it uses hash aggregation with group-by on recipe ($O(N)$), and Neo4j must traverse every relation in the graph and the traversal step overhead of Neo4j is greater than the Postgres tuple comparison.

Results: variable length query result

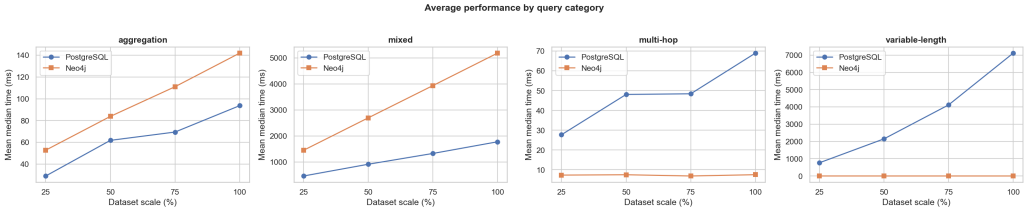


Results: variable length query analysis

- **Reachable recipes in N-hops**: here Neo4j wins by orders of magnitude (sub ms vs 7000+ ms at full scale). This is the canonical showcase for graph dbs. The query counts distinct recipes reachable within N (in this case 2 because otherwise Postgres was too slow) via shared ingredients.
  Postgres must implement BFS manually using a recursive CTE and each recursive step creates intermediate results which grow with data size.
  Neo4j has a **native variable-length path operator** (`HAS_INGREDIENT*1..N`) that maintains a compact visited set and follows relationship pointers without materializing intermediate results, pruining already visited nodes in $O(1)$.
  The reachability answer is bounded by graph topology and not global dataset size.

Results: Average Performance by Query Category

6 Experimental Analysis: Recipes Dataset



Neo4j worse | **Neo4j much worse** | **Neo4j much better** | **Neo4j much better**

Table of Contents

7 Conclusions

- Data Model, Schema & Storage Layout
- Query Languages: SQL vs. Cypher
- Query Complexity Analysis
- Consistency, ACID & the CAP Theorem
- Scaling
- Experimental Analysis: Recipes Dataset
- Conclusions

When to Choose a Relational Database

7 Conclusions

Choose a Relational DB when:

- Data is **tabular and well-structured** with a stable schema
- Workload is **transactional** (banking, e-commerce, ERP)
- Queries involve **global aggregations** or OLAP analytics
- Joins are **shallow** (1–2 levels)

Choose a Graph DB when:

- Data is **highly connected** and relationships are central
- Queries involve **multi-hop traversals** (fraud rings, social networks, supply chains)
- **Pathfinding** or **reachability** is required
- Schema evolves frequently (new relationship types)
- **Recommendation engines**
- **Knowledge graphs** or semantic data